

CRYPTOGRAPHY

POONGUZHALI N

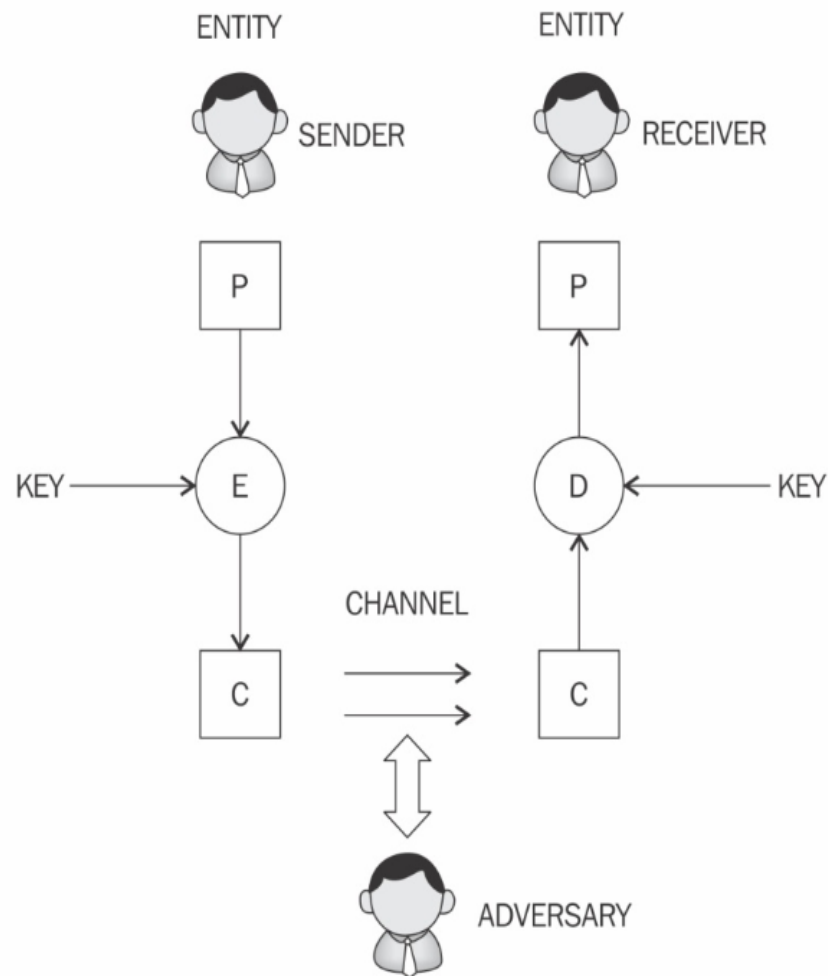
SCHOOL OF COMPUTER & SYSTEMS SCIENCES

JAWAHARLAL NEHRU UNIVERSITY

Cryptography

- Cryptography is the science of making information secure
- Ciphers are algorithms used to encrypt or decrypt data so that if intercepted by an adversary.
- Data is meaningless to them without decryption which requires a secret key.
- Cryptography is used to provide a confidentiality, integrity, authentication and non-repudiation and accountability.

A GENERIC CRYPTOGRAPHY MODEL



Entity: Either a person or system that sends, receives, or performs operations on data

Sender: An entity that transmits the data

Receiver: An entity that takes delivery of the data

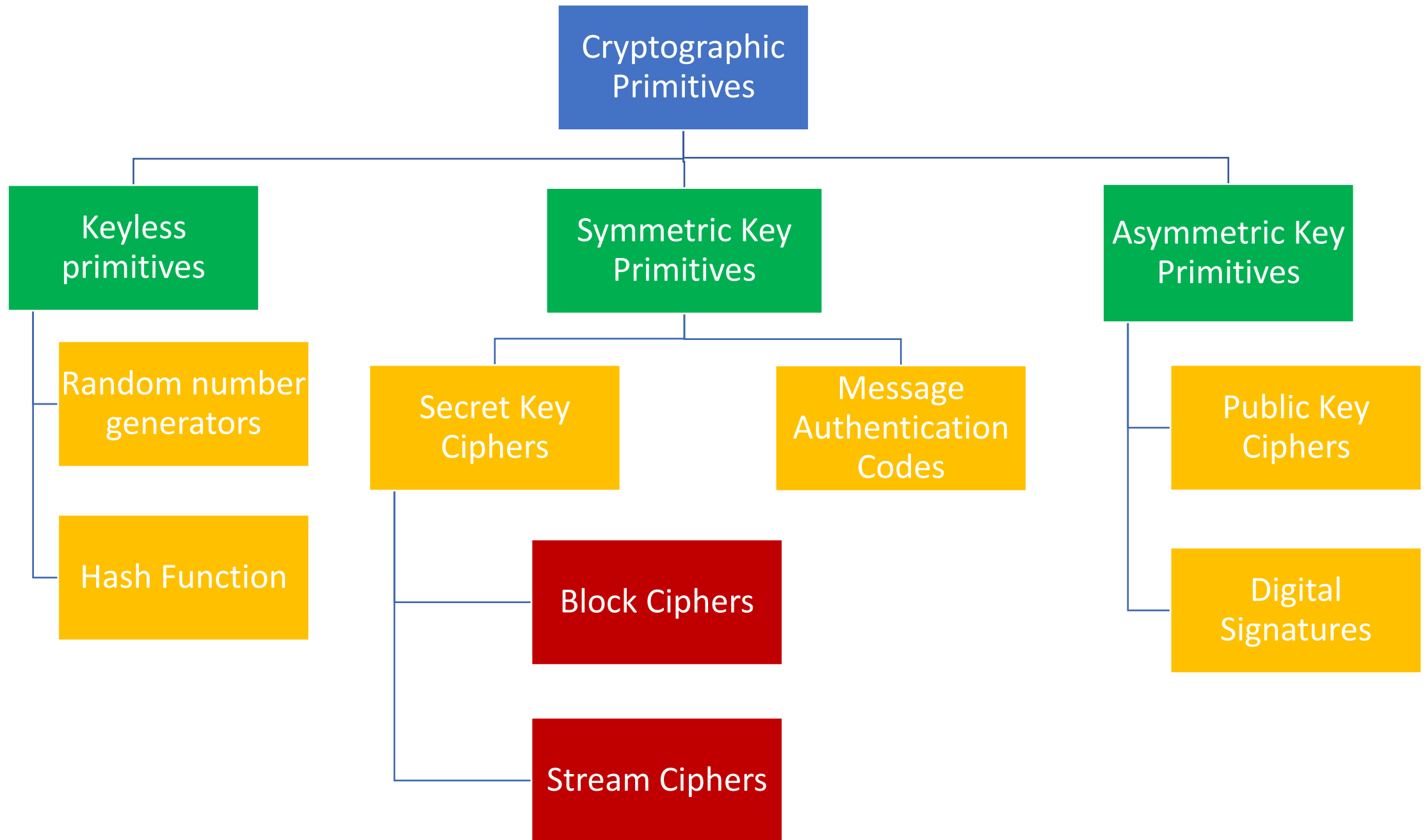
Adversary: This is an entity that tries to circumvent the security service

Key: A key is data that is used to encrypt or decrypt other data

Channel: Channel provides a medium of communication between entities

FEATURES OF CRYPTOGRAPHY

- Confidentiality: The assurance that information is only available to authorized entities.
- Integrity: The assurance that information is modifiable only by authorized entities
- Authentication: provides assurance about the identity of an entity or the validity of a message, entity authentication and data origin authentication.
- Non-repudiation: an entity cannot deny a previous commitment or action by providing incontrovertible evidence
- Accountability: the assurance which states that actions affecting security can be traced back to the responsible party. This is usually provided by logging and audit mechanisms in systems where a detailed audit is required due to the nature of the business



Keyless Primitives

- Random Number Generators : They are software or hardware systems that make use of randomness available in the real world.
- Pseudorandom Number Generators: They are deterministic functions that work on the principle of using random initial value called a seed to produce a random-looking set of elements.
- Hash Functions: They are used to create fixed-length digests of arbitrarily long input strings. Hash functions are keyless and they provide a data integrity service. They are usually built using iterated and dedicated hash function construction techniques.



Original Data



Hash Function



Hash Value/Digest

Properties of Hash Functions

- Deterministic : Given an input it will always produce the same output
- Fixed size output: Compression of arbitrary message into fixed-length digests
- Non-reversible: Impossible to determine the input that produced a given output
- Computationally efficient: Easy to Compute and speed
- One way function: Pre-image resistance
- Collision Resistance: A collision in a hash function occurs when two different input values produce the same hash value. weak collision resistance and strong collision resistance

Types of Hash Functions

- Cryptographic: These are secure and resistant to various attacks, such as collision attacks, pre-image attacks, and second pre-image attacks.
- Non-cryptographic: These are not secure but applicable in data indexing, checksum generation, and error detection.
- Key based: These require a secret key in addition to the input data. The private key help prevent attacks from malicious users who attempt to manipulate the input data to produce a specific output. HMAC (Hash-based Message Authentication Code)
- Iterated: These apply a compression function repeatedly to the input data to produce the final hash value.

Cryptographic Hash Functions

- SHA-0: 160 bits
- SHA-1: 160 bits
- SHA-2: 224,256,384,512
- SHA -3:

SHA 256

- Secure Hash Algorithm. Published in 2001, it was a joint effort between the NSA and NIST to introduce a successor to the SHA 1 family, which was slowly losing strength against brute force attacks.
- By using bitwise operations, modular additions, and compression functions, the hashing algorithm reduces the input data into a smaller form that is impossible to comprehend.

Step one: Padding Bits

- The total length of our message must be a multiple of 512
- In this step, bits are appended to the end of the message such that the final length of the message must be 64 bits less than a multiple of 512.
- The formula below depicts this step:
 - $m+p=(n*512)-64$
- where m = length of the message, p = length of the padding, and n = a constant.

Step 1: Pre-Processing

- Convert the message / data to binary form, eg “hello world”

```
01101000 01100101 01101100 01101100 01101111 00100000 01110111  
01101111  
01110010 01101100 01100100
```

- Append “1” to the binary data

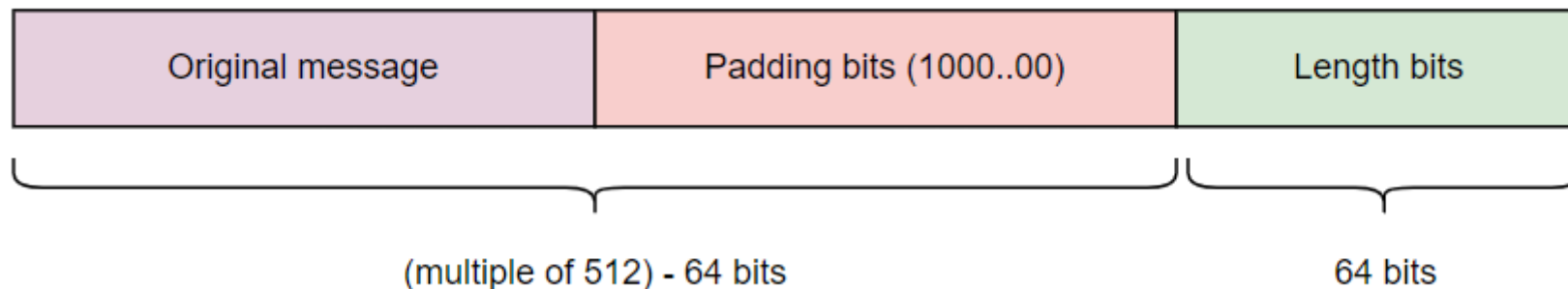
```
01101000 01100101 01101100 01101100 01101111 00100000 01110111  
01101111  
01110010 01101100 01100100 1
```

- Pad with 0's until data is a multiple of 512, less 64 bits (448 bits in our case):

[illegible]

Step Two: Padding Length

Next, we take the modulus of the original message with 2^{32} to get 64 bits of data. Appending this to the padded message makes our processed message an exact multiple of 512



Append 64 bits to the end, where the 64 bits are a big-endian integer representing the length of the original input in binary. In our case, 88, or in binary, “1011000”.

01101000	01100101	01101100	01101100	01101111	00100000	01110111
01101111						
01110010	01101100	01100100	10000000	00000000	00000000	00000000
00000000						
00000000	00000000	00000000	00000000	00000000	00000000	00000000
00000000						
00000000	00000000	00000000	00000000	00000000	00000000	00000000
00000000						
00000000	00000000	00000000	00000000	00000000	00000000	00000000
00000000						
00000000	00000000	00000000	00000000	00000000	00000000	00000000
00000000						
00000000	00000000	00000000	00000000	00000000	00000000	00000000
00000000						
00000000	00000000	00000000	00000000	00000000	00000000	00000000
00000000						
00000000	00000000	00000000	00000000	00000000	00000000	00000000
01011000						

Step 3: **Buffer initialization**

- Before we begin carrying out computations on the message, we need to initialize some buffer values. The default eight buffer values are shown below. These are hard-coded constants representing hash values.

```
A = 0x6a09e667
B = 0xbb67ae85
C = 0x3c6ef372
D = 0xa54ff53a
E = 0x510e527f
F = 0x9b05688c
G = 0x1f83d9ab
H = 0x5be0cd19
```

```
k[0..63] =
0x428a2f98 0x71374491 0xb5c0fbcf 0xe9b5dba5 0x3956c25b
0x59f111f1 0x923f82a4 0xab1c5ed5 0xd807aa98 0x12835b01
0x243185be 0x550c7dc3 0x72be5d74 0x80deb1fe 0x9bdc06a7
0xc19bf174 0xe49b69c1 0xefbe4786 0x0fc19dc6 0x240ca1cc
0x2de92c6f 0x4a7484aa 0x5cb0a9dc 0x76f988da 0x983e5152
0xa831c66d 0xb00327c8 0xbf597fc7 0xc6e00bf3 0xd5a79147
0x06ca6351 0x14292967 0x27b70a85 0x2e1b2138 0x4d2c6dfc
0x53380d13 0x650a7354 0x766a0abb 0x81c2c92e 0x92722c85
0xa2bfe8a1 0xa81a664b 0xc24b8b70 0xc76c51a3 0xd192e819
0xd6990624 0xf40e3585 0x106aa070 0x19a4c116 0x1e376c08
0x2748774c 0x34b0bcb5 0x391c0cb3 0x4ed8aa4a 0x5b9cca4f
0x682e6ff3 0x748f82ee 0x78a5636f 0x84c87814 0x8cc70208
0x90bffffa 0xa4506ceb 0xbef9a3f7 0xc67178f2
```

Initialize	Round	Constants	(k)
------------	-------	-----------	-----

- Similar to step 2, we are creating some constants. This time, there are 64 of them. Each value (0–63) is the first 32 bits of the fractional parts of the cube roots of the first 64 primes (2–311).

0x428a2f98	0x71374491	0xb5c0fbcf	0xe9b5dba5	0x3956c25b	0x59f111f1
0x923f82a4	0xab1c5ed5				
0xd807aa98	0x12835b01	0x243185be	0x550c7dc3	0x72be5d74	0x80deb1fe
0x9bdc06a7	0xc19bf174				
0xe49b69c1	0xefbe4786	0x0fc19dc6	0x240ca1cc	0x2de92c6f	0x4a7484aa
0x5cb0a9dc	0x76f988da				
0x983e5152	0xa831c66d	0xb00327c8	0xbf597fc7	0xc6e00bf3	0xd5a79147
0x06ca6351	0x14292967				
0x27b70a85	0x2e1b2138	0x4d2c6dfc	0x53380d13	0x650a7354	0x766a0abb
0x81c2c92e	0x92722c85				
0xa2bfe8a1	0xa81a664b	0xc24b8b70	0xc76c51a3	0xd192e819	0xd6990624
0xf40e3585	0x106aa070				
0x19a4c116	0x1e376c08	0x2748774c	0x34b0bcb5	0x391c0cb3	0x4ed8aa4a
0x5b9cca4f	0x682e6ff3				
0x748f82ee	0x78a5636f	0x84c87814	0x8cc70208	0x90bffffa	0xa4506ceb
0xbef9a3f7	0xc67178f2				

Chuck Loop

- **For i from w[16...63]:**
- $s0 = (w[i-15] \text{ rightrotate } 7) \text{ xor } (w[i-15] \text{ rightrotate } 18) \text{ xor } (w[i-15] \text{ rightshift } 3)$
- $s1 = (w[i-2] \text{ rightrotate } 17) \text{ xor } (w[i-2] \text{ rightrotate } 19) \text{ xor } (w[i-2] \text{ rightshift } 10)$
- $w[i] = w[i-16] + s0 + w[i-7] + s1$
- **Let's do w[16] so we can see how it works:**

w[1] rightrotate 7:

01101111001000000111011101101111 ->
11011110110111100100000011101110

w[1] rightrotate 18:

01101111001000000111011101101111 ->
00011101110110111101101111001000

w[1] rightshift 3:

01101111001000000111011101101111 ->
00001101111001000000111011101101

s0 = 11011110110111100100000011101110 XOR
00011101110110111101101111001000 XOR 00001101111001000000111011101101

s0 = 11001110111000011001010111001011

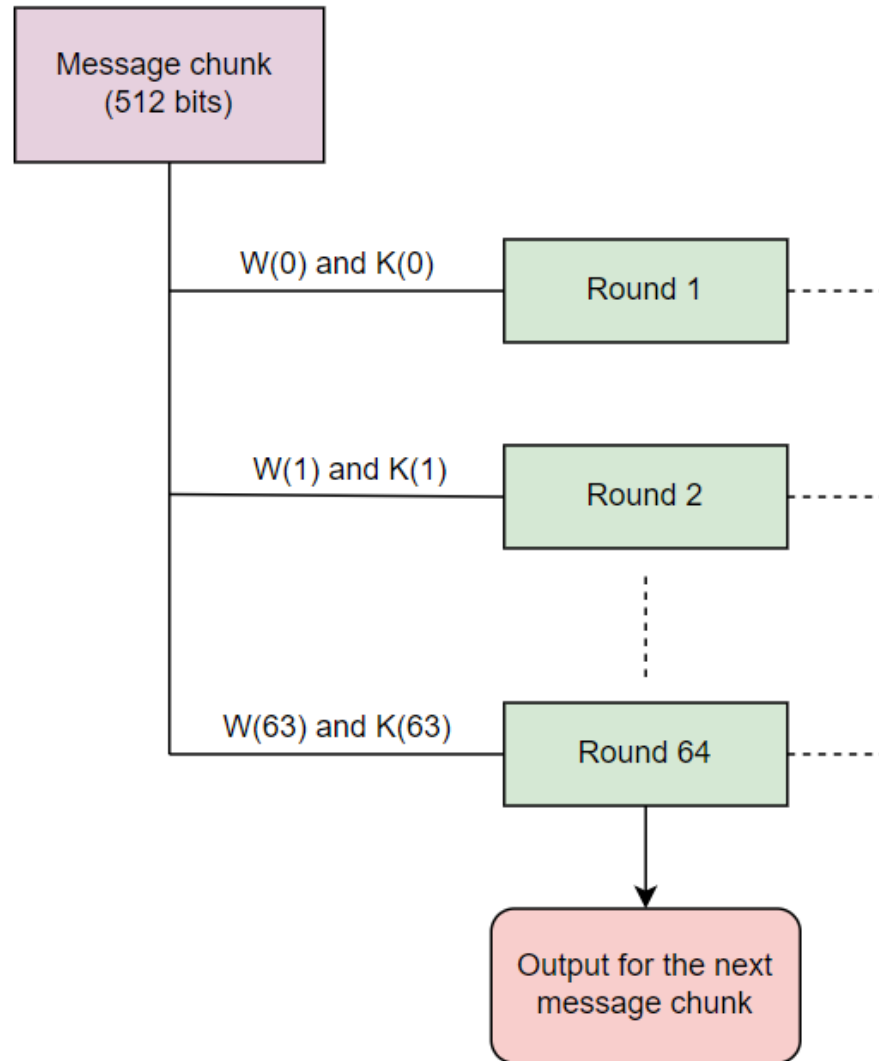

```
w[16] = w[0] + s0 + w[9] + s1
```

```
w[16] = 01101000011001010110110001101100 +  
11001110111000011001010111001011 + 00000000000000000000000000000000 +  
00000000000000000000000000000000
```

```
// addition is calculated modulo 2^32
```

```
w[16] = 001101110100011100000001000110111
```

STEP 4: COMPRESSION FUNCTION



Compression

- Initialize variables a, b, c, d, e, f, g, h and set them equal to the current hash values respectively. $h_0, h_1, h_2, h_3, h_4, h_5, h_6, h_7$
- Run the compression loop. The compression loop will mutate the values of $a...h$. The compression loop is as follows:
 - for i from 0 to 63
 - $S1 = (e \text{ rightrotate } 6) \text{ xor } (e \text{ rightrotate } 11) \text{ xor } (e \text{ rightrotate } 25)$
 - $ch = (e \text{ and } f) \text{ xor } ((\text{not } e) \text{ and } g)$
 - $\text{temp1} = h + S1 + ch + k[i] + w[i]$
 - $S0 = (a \text{ rightrotate } 2) \text{ xor } (a \text{ rightrotate } 13) \text{ xor } (a \text{ rightrotate } 22)$
 - $\text{maj} = (a \text{ and } b) \text{ xor } (a \text{ and } c) \text{ xor } (b \text{ and } c)$
 - $\text{temp2} := S0 + \text{maj}$

`h = g`

`g = f`

`e = d + temp1`

`d = c`

`c = b`

`b = a`

`a = temp1 + temp2`

That entire calculation is done 63 more times, modifying the variables a-h throughout.

```
h0 = 6A09E667 = 01101010000010011110011001100111
h1 = BB67AE85 = 10111011011001111010111010000101
h2 = 3C6EF372 = 00111100011011101111001101110010
h3 = A54FF53A = 10100101010011111111010100111010
h4 = 510E527F = 01010001000011100101001001111111
h5 = 9B05688C = 10011011000001010110100010001100
h6 = 1F83D9AB = 00011111100000111101100110101011
h7 = 5BE0CD19 = 01011011111000001100110100011001
```

```
a = 4F434152 = 001001111010000110100000101010010
b = D7E58F83 = 011010111111001011000111110000011
c = 68BF5F65 = 001101000101111110101111101100101
d = 352DB6C0 = 000110101001011011011011011000000
e = 73769D64 = 001110011011101101001110101100100
f = DF4E1862 = 011011111010011100001100001100010
g = 71051E01 = 001110001000001010001111000000001
h = 870F00D0 = 0100001110000111100000000011010000
```

Modify final values

- After the compression loop, but still, within the chunk loop, modify the hash values by adding their respective variables to them, a-h. As usual, all

```
h0 = h0 + a = 10111001010011010010011110111001
h1 = h1 + b = 10010011010011010011111000001000
h2 = h2 + c = 10100101001011100101001011010111
h3 = h3 + d = 1101101001111101101010111111010
h4 = h4 + e = 1100010010000100111011111100011
h5 = h5 + f = 01111010010100111000000011101110
h6 = h6 + g = 10010000100010001111011110101100
h7 = h7 + h = 11100010111011111100110111101001
```

Concatenate Final Hash

```
digest = h0 append h1 append h2 append h3 append h4 append h5 append  
h6 append h7  
=  
B94D27B9934D3E08A52E52D7DA7DABFAC484EFE37A5380EE9088F7ACE2EFCDE9
```

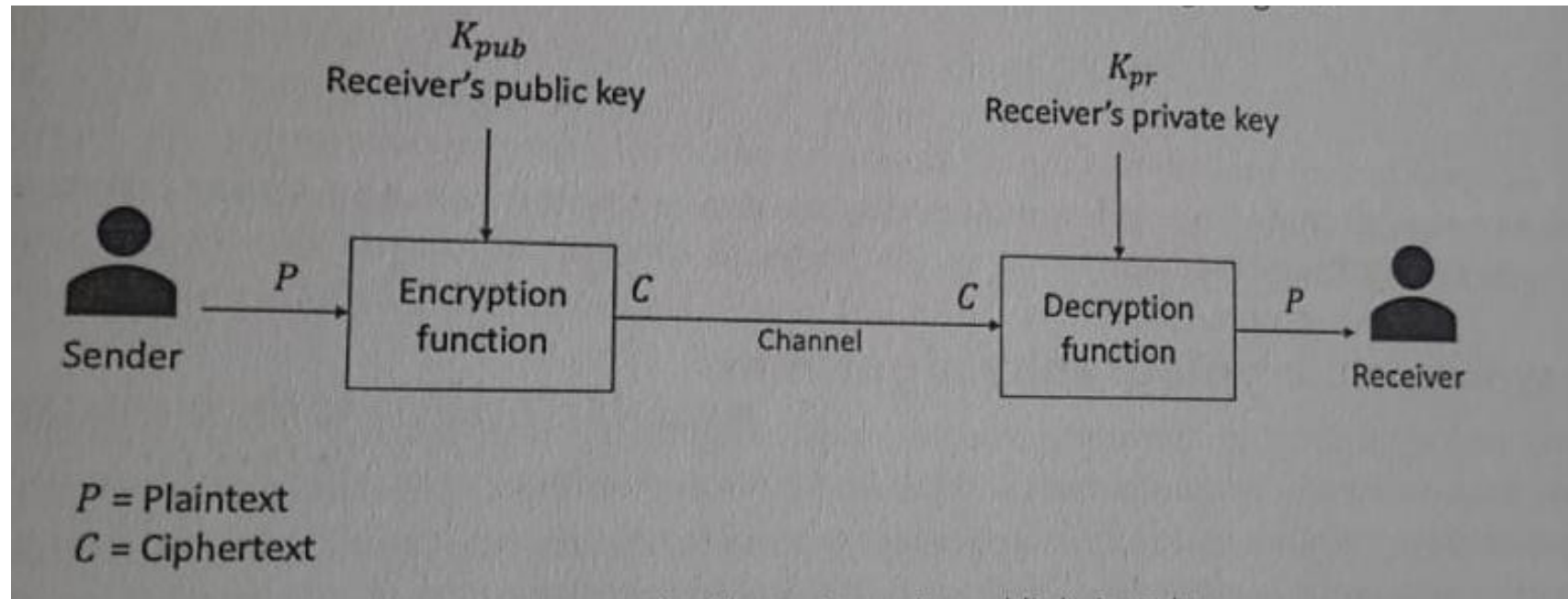
Symmetric cyptography Key

- The key that is used to encrypt the data is the same one that is used for decrypting the data.
- Also called as shared key cryptography
- Ephemeral key - only for short period
- Static key – indented for long term

Asymmetric Cryptography

- A type of cryptography where the key that is used to encrypt the data is different from the key that is used to decrypt the data.
- Asymmetric cryptography is also known as public key cryptography.
- A public key as the name suggest is freely available and published by the private key owner.
- This process includes authenticity and identification of the publisher of the public keys.

Encryption and Decryption using Public Key

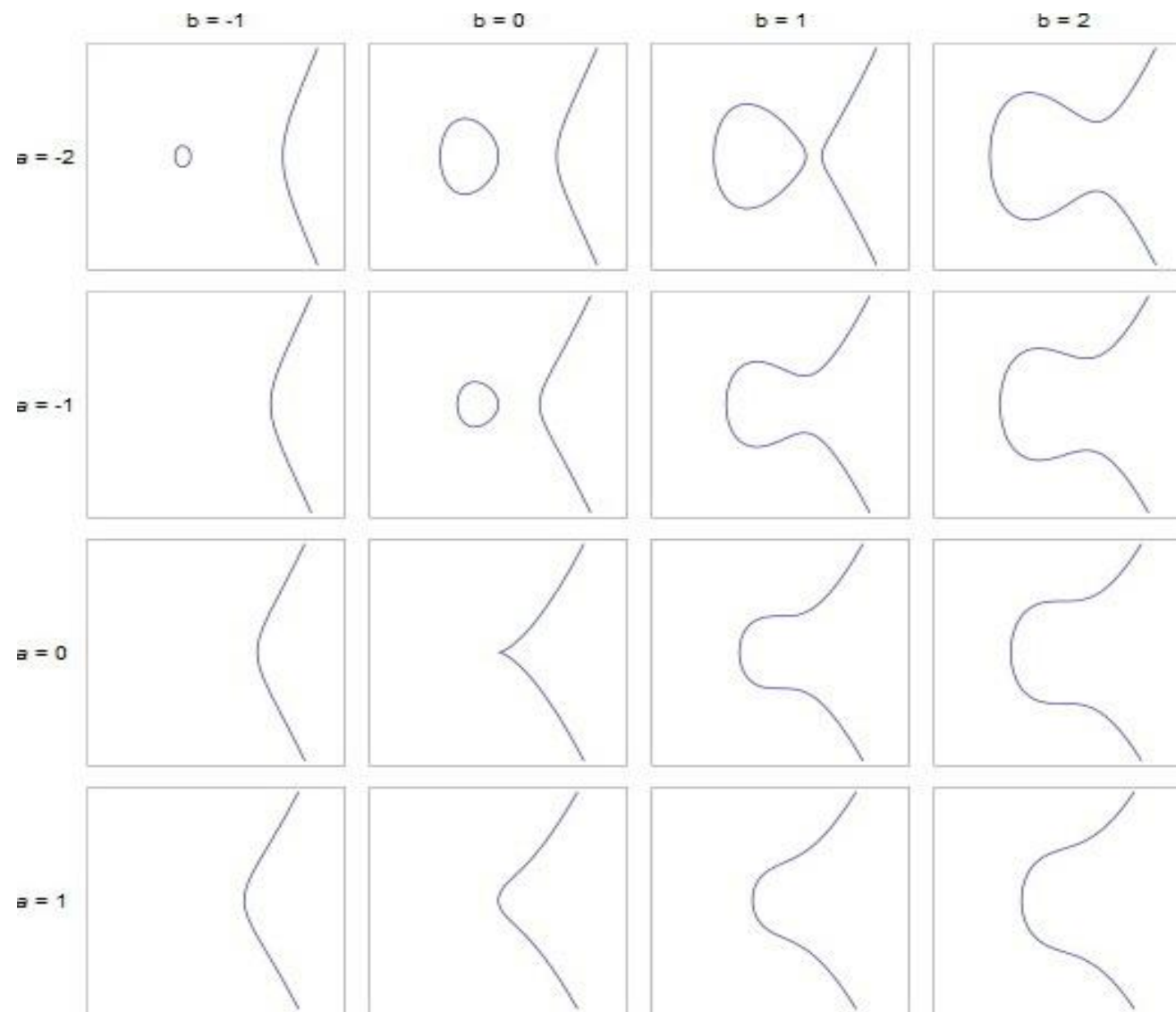


Asymmetric cryptography Algorithms

- Integer factorization: Schemes are based on hard problem that large integers are extreme to factor.
- Discrete logarithm: based on modular arithmetic and easy to calculate but computationally impractical to find the exponent of the generator.
- Elliptic Curves: Are based on discrete logarithm but based on elliptic curve.

Introduction

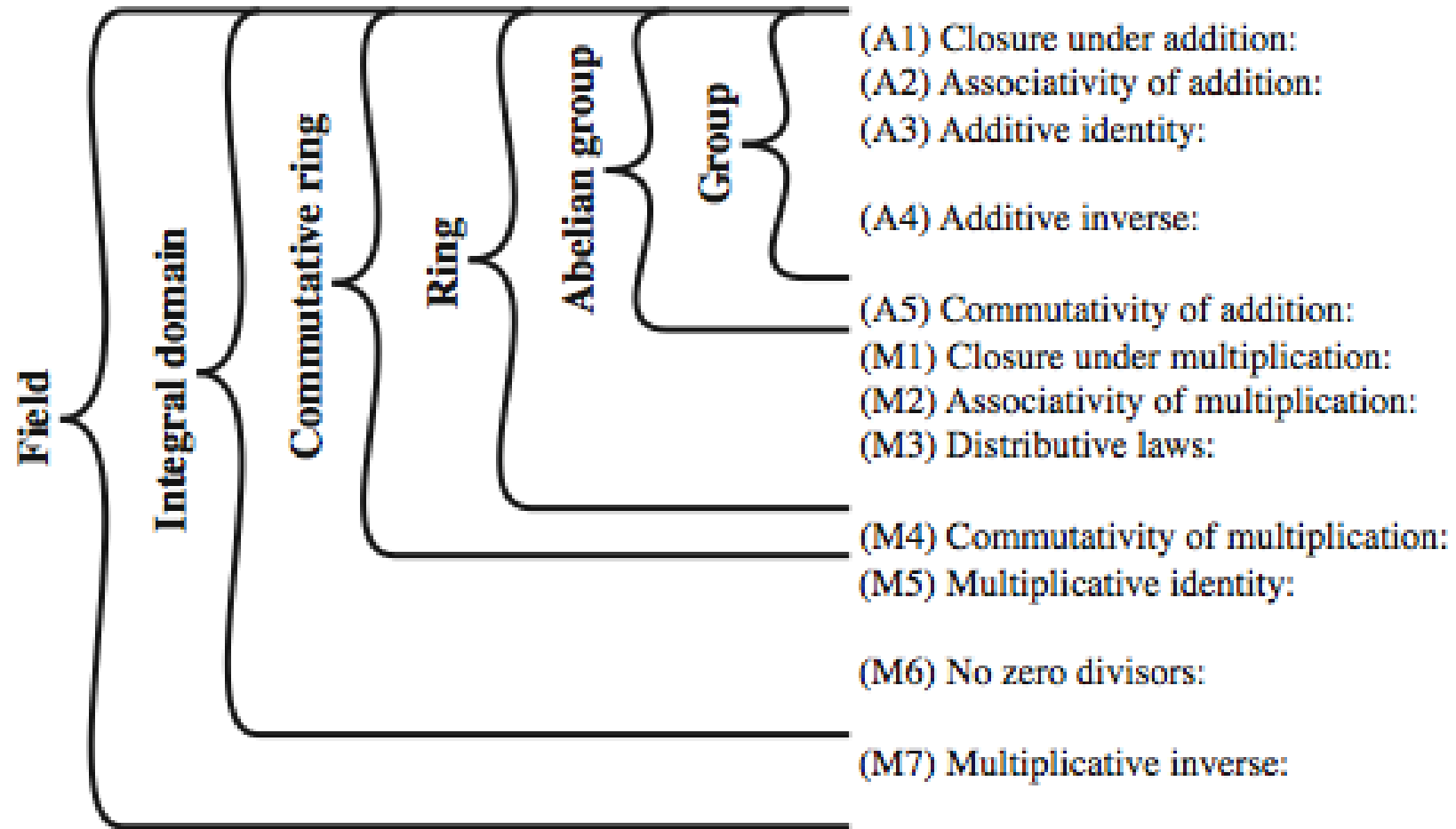
- What are Elliptic Curves?
 - Curve with standard form $y^2 = x^3 + ax + b$ $a, b \in \mathbb{R}$
- Characteristics of Elliptic Curve
 - Forms an abelian group
 - Symmetric about the x-axis
 - *Point at Infinity* acting as the identity element



Examples of Elliptic Curves

Finite Fields

- aka Galois Field
- $GF(p^n)$ = a set of integers $\{0, 1, 2, \dots, p^n - 1\}$
where p is a prime, n is a positive integer
- It is denoted by $\{F, +, \times\}$
where $+$ and $\times$ are the group operators



Group, Ring, Field

Why Elliptic Curve Cryptography?

- Shorter Key Length
- Lesser Computational Complexity
- Low Power Requirement
- More Secure

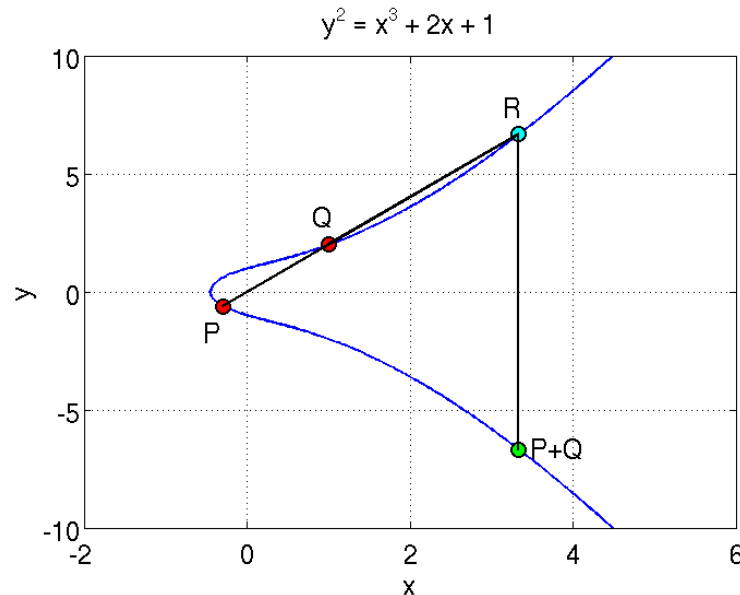
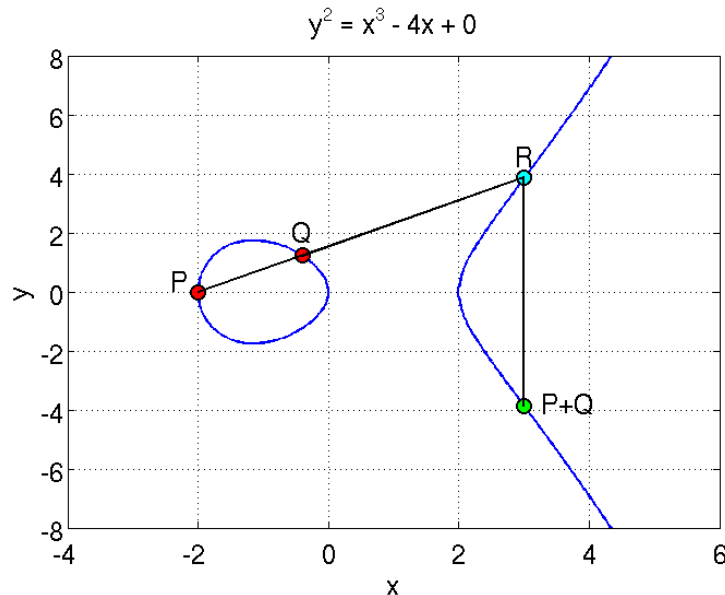
Comparable Key Sizes for Equivalent Security

Symmetric Encryption (Key Size in bits)	RSA and Diffie-Hellman (modulus size in bits)	ECC Key Size in bits
56	512	112
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	512

What is Elliptic Curve Cryptography?

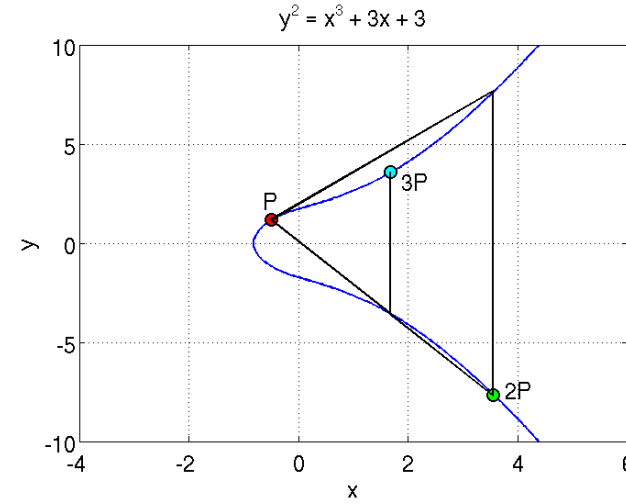
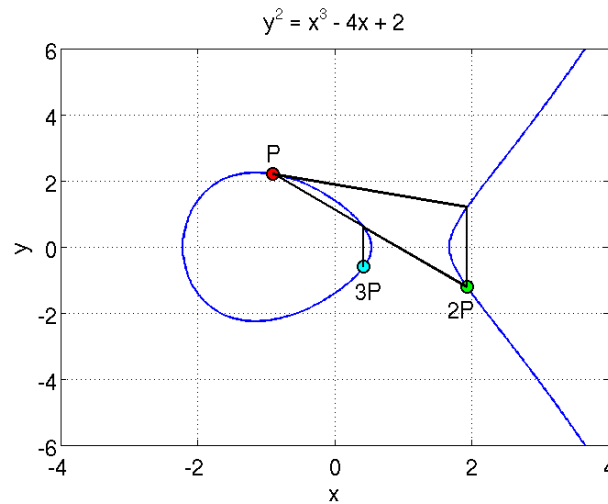
- Implementing Group Operations
 - Main operations - point addition and point multiplication
 - Adding two points that lie on an Elliptic Curve – results in a third point on the curve
 - Point multiplication is repeated addition
 - If P is a known point on the curve (aka Base point; part of domain parameters) and it is multiplied by a scalar k , $Q=kP$ is the operation of adding $P + P + P + P \dots + P$ (k times)
 - Q is the resulting public key and k is the private key in the public-private key pair

What is Elliptic Curve Cryptography?



- Adding two points on the curve
- P and Q are added to obtain P+Q which is a reflection of R along the X axis

What is Elliptic Curve Cryptography?



- A tangent at P is extended to cut the curve at a point; its reflection is 2P
- Adding P and 2P gives 3P
- Similarly, such operations can be performed as many times as desired to obtain $Q = kP$

What is Elliptic Curve Cryptography?

- Discrete Log Problem

- The security of ECC is due the intractability or difficulty of solving the inverse operation of finding k given Q and P
- This is termed as the discrete log problem
- Methods to solve include brute force and Pollard's Rho attack both of which are computationally expensive or unfeasible
- The version applicable in ECC is called the Elliptic Curve Discrete Log Problem
- Exponential running time

ECC in Windows DRM v2.0

A Practical Example :

Finite field chosen

$p = 785963102379428822376694789446897396207498568951$

$G_x = 771507216262649826170648268565579889907769254176$

$G_y = 390157510246556628525279459266514995562533196655$

$y^2 = x^3 + 317689081251325503476317476413827693272746955927x + 790528966078787587181205720257185354321100651934$

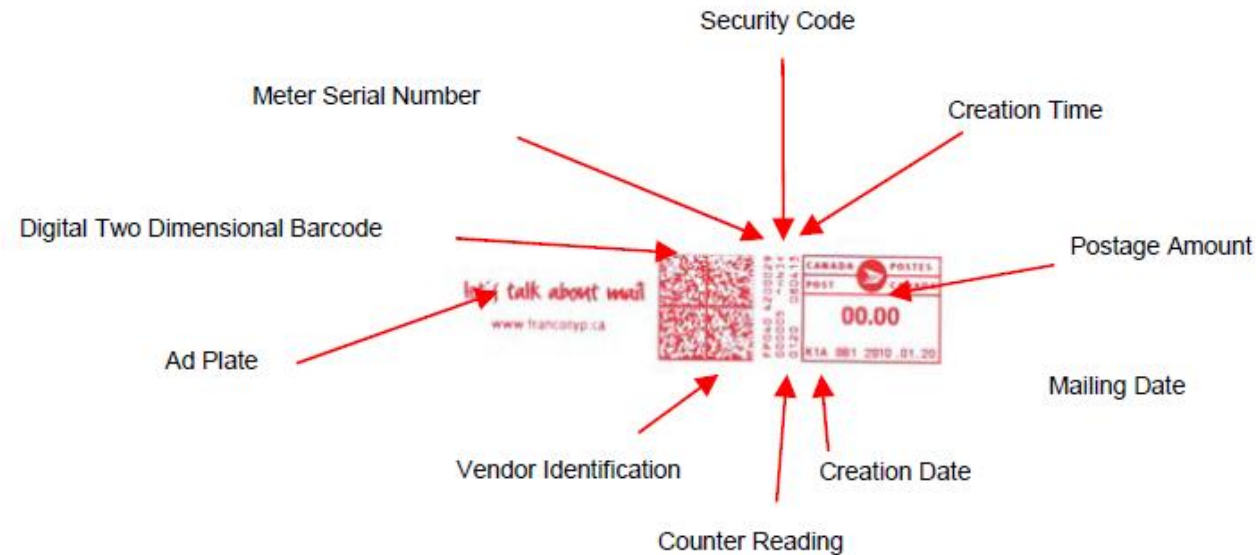
G_x and G_y constitute the agreed upon base point (P) and the numbers in the above equation are values for the parameters a and b

Elliptic Curve Schemes

- Elliptic Curve Digital Signature Algorithm (ECDSA)
- Elliptic Curve Pintsov Vanstone Signature(ECPVS)
- Elliptic Curve Diffie-Hellman (ECDH)

Elliptic Curve Digital Signature Algorithm (ECDSA)

- Elliptic curve variant of Digital Signature Algorithm



Canadian postage stamp that uses ECDSA

ECDSA

- Signature Generation

Once we have the domain parameters and have decided on the keys to be used, the signature is generated by the following steps.

1. A random number k , $1 \leq k \leq n-1$ is chosen
2. $kG = (x_1, y_1)$ is computed. x_1 is converted to its corresponding integer x_1'
3. Next, $r = x_1 \bmod n$ is computed
4. We then compute $k^{-1} \bmod q$
5. $e = \text{HASH}(m)$ where m is the message to be signed
6. $s = k^{-1}(e + dr) \bmod n$

where d is the private key of the sender.

We have the signature as (r,s)

ECDSA

- Signature Verification

At the receiver's end the signature is verified as follows:

1. Verify whether r and s belong to the interval $[1, n-1]$ for the signature to be valid.
2. Compute $e = \text{HASH}(m)$. The hash function should be the same as the one used for signature generation.
3. Compute $w = s^{-1} \bmod n$.
4. Compute $u_1 = ew \bmod n$ and $u_2 = rw \bmod n$.
5. Compute $(x_1, y_1) = u_1G + u_2Q$.
6. The signature is valid if $r = x_1 \bmod n$, invalid otherwise.

This is how we know that the verification works the way we want it to:

We have, $s = k^{-1}(e + dr) \bmod n$ which we can rearrange to obtain, $k = s^{-1}(e + dr)$ which is

$$s^{-1}e + s^{-1}rd$$

Elliptic Curve Pintsov Vanstone Signature (ECPVS)

- Signature scheme using Elliptic Curves
- More efficient than RSA as overhead is extremely low

ECPVS

- Signature Generation

The plaintext message is split into two parts: part C representing the data elements requiring confidentiality and part V representing the data elements presented in plaintext. Both the parts are signed. The signature is generated as follows:

1. A random number k , $1 \leq k \leq n-1$ is chosen.
2. Calculate the point R on the curve ($R = kG$).
3. Use point R and a symmetric encryption algorithm to get $e = T_R(C)$.
4. Calculate a variable d such that $d = \text{HASH}(e || I_A || V)$

where I_A is the identity of the mailer terminal.

5. Now calculate the other part of the signature s as follows: $s = ad + k(\text{mod } n)$; where a is the private key of the sender.

The signature pair (s,e) is transmitted together with the portion V of the plaintext.

ECPVS

- Signature Verification

1. Retrieve Q_A (Q_A is mailer A's public key)
2. Calculate the variable $d = \text{HASH}(e || I_A || V)$ using the same HASH algorithm as the one used for generating the signature.
3. Compute $U = sG - dQ_A$.
4. Recover $C = T_u^{-1}(e)$.
5. Run a redundancy test on C. If the test fails, discard the message. Else, the plaintext is recovered.

We have, $s = ad + k$. Multiply by base point G to obtain $sG = adG + kG$ which is $dQ_A + R$
Therefore, $R = sG - dQ_A$ which is U . Comparing the decrypted versions, m and m' obtained using U and R , we ascertain the validity of the signature

Elliptic Curve Diffie-Hellman (ECDH)

- Elliptic curve variant of the key exchange Diffie-Hellman protocol.
- Decide on domain parameters and come up with a Public/Private key pair
- To obtain the private key, the attacker needs to solve the discrete log problem

ECDH

- How the key exchange takes place:
 1. Alice and Bob publicly agree on an elliptic curve E over a large finite field F and a point P on that curve.
 2. Alice and Bob each privately choose large random integers, denoted a and b
 3. Using elliptic curve point-addition, Alice computes aP on E and sends it to Bob. Bob computes bP on E and sends it to Alice.
 4. Both Alice and Bob can now compute the point abP Alice by multiplying the received value of bP by her secret number a and Bob vice-versa.
 5. Alice and Bob agree that the x coordinate of this point will be their shared secret value.

Pros and Cons

- Pros
 - Shorter Key Length
 - Same level of security as RSA achieved at a much shorter key length
 - Better Security
 - Secure because of the ECDLP
 - Higher security per key-bit than RSA
 - Higher Performance
 - Shorter key-length ensures lesser power requirement – suitable in wireless sensor applications and low power devices
 - More computation per bit but overall lesser computational expense or complexity due to lesser number of key bits

Pros and Cons

- Cons
 - Relatively newer field
 - Idea prevails that all the aspects of the topic may not have been explored yet – possibly unknown vulnerabilities
 - Doesn't have widespread usage
 - Not perfect
 - Attacks still exist that can solve ECC (112 bit key length has been publicly broken)
 - Well known attacks are the Pollard's Rho attack (complexity $O(\sqrt{n})$), Pohlig's attack, Baby Step, Giant Step etc